Part C: Regression Analysis

QUESTION 21: Perform an exploratory data analysis on the provided Diamonds dataset. Report the following:

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df =
pd.read_csv("/content/drive/MyDrive/ece219-project3/diamonds_ece219.csv")
# df = pd.read_csv("diamonds.csv")

print(df.shape)
print(df.head())
print(df.dtypes)
```

```
(149871, 15)
   Unnamed: 0 color clarity  carat        cut   symmetry     polish  \
0           0     E    VVS2   0.09  Excellent  Very Good  Very Good
1           1     E    VVS2   0.09  Very Good  Very Good  Very Good
2           2     E    VVS2   0.09  Excellent  Very Good  Very Good
3           3     E    VVS2   0.09  Excellent  Very Good  Very Good
4           4     E    VVS2   0.09  Very Good  Very Good  Excellent

   depth_percent  table_percent  length  width  depth girdle_min
girdle_max  \
0           62.7           59.0    2.85   2.87   1.79          M
M
1           61.9           59.0    2.84   2.89   1.78        STK
STK
2           61.1           59.0    2.88   2.90   1.77         TN
M
3           62.0           59.0    2.86   2.88   1.78          M
STK
4           64.9           58.5    2.79   2.83   1.82        STK
```

```
STK

   price
0    200
1    200
2    200
3    200
4    200
Unnamed: 0          int64
color              object
clarity            object
carat             float64
cut                object
symmetry           object
polish             object
depth_percent     float64
table_percent     float64
length            float64
width             float64
depth             float64
girdle_min         object
girdle_max         object
price               int64
dtype: object
```

```
df = df.drop(columns=['Unnamed: 0'])
numeric_cols = df.select_dtypes(include=np.number).columns
cat_cols = df.select_dtypes(include="object").columns

print("Numeric columns:", list(numeric_cols))
print("Categorical columns:", list(cat_cols))
```

```
Numeric columns: ['carat', 'depth_percent', 'table_percent', 'length',
'width', 'depth', 'price']
Categorical columns: ['color', 'clarity', 'cut', 'symmetry', 'polish',
'girdle_min', 'girdle_max']
```

QUESTION 21: Perform an exploratory data analysis on the provided Diamonds dataset.

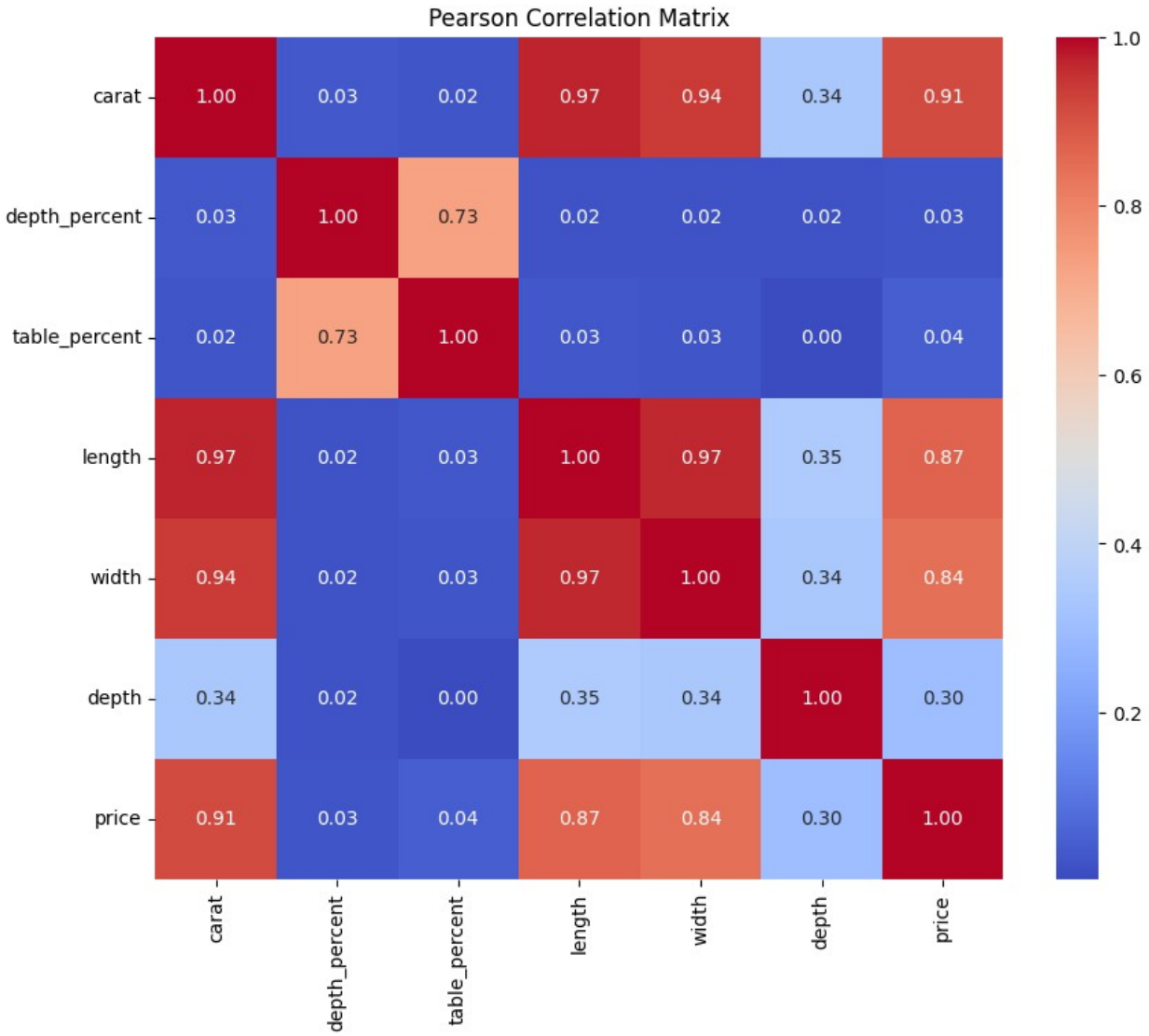
Correlation Analysis: Plot a heatmap of the Pearson correlation matrix. Report which features have the highest absolute correlation with the target variable (price). Briefly describe what the correlation patterns suggest.

```
corr = df[numeric_cols].corr()

plt.figure(figsize=(10, 8))
```

```
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Pearson Correlation Matrix")
plt.show()

price_corr = corr["price"].abs().sort_values(ascending=False)
print("Absolute correlation with price:")
print(price_corr)
```



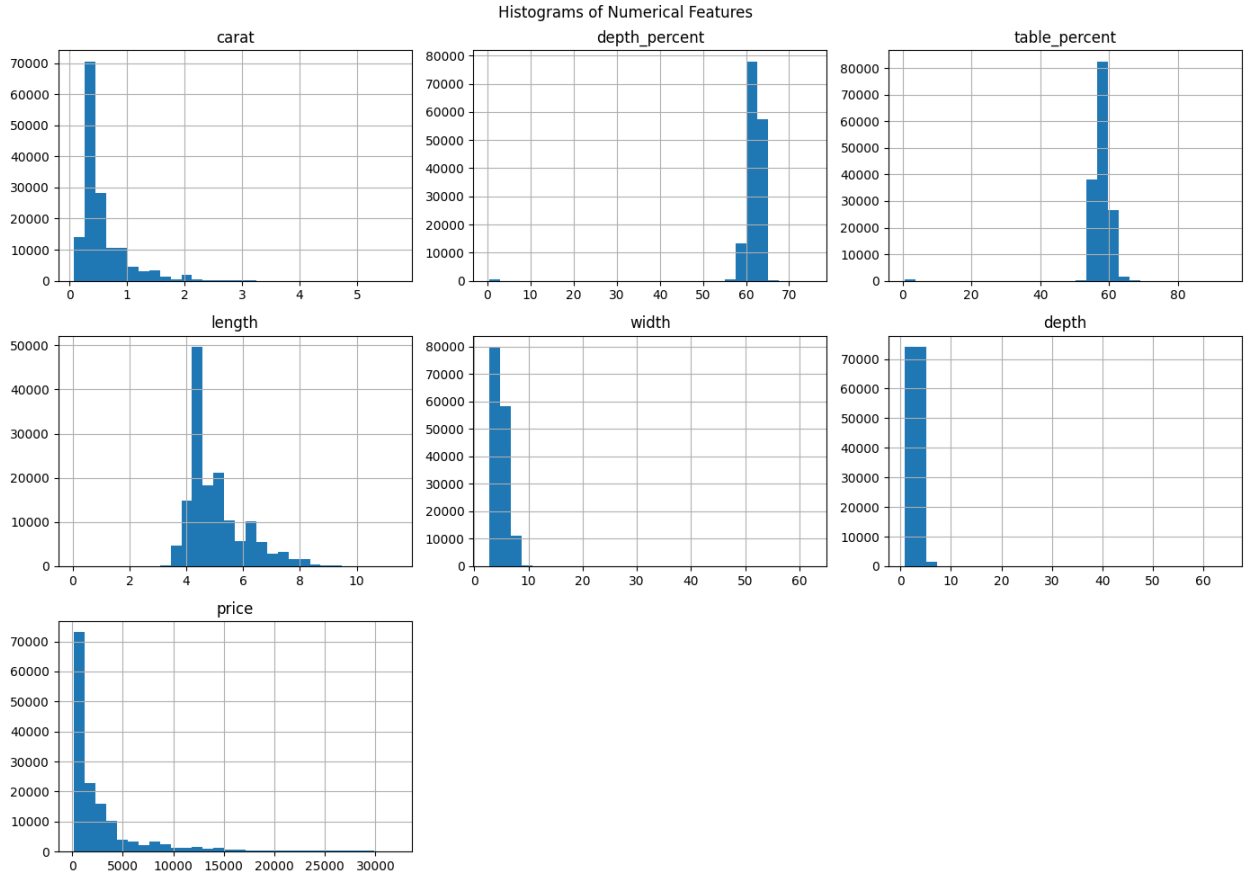

```
Absolute correlation with price:
price            1.000000
carat            0.913479
length           0.869521
width            0.841887
depth            0.299696
table_percent    0.042453
```

```
depth_percent    0.025469
Name: price, dtype: float64
```

The feature carat has the highest correlation with price, suggesting that diamond weight strongly influences value. Physical dimensions such as length and width also correlate with price because they reflect the diamond's size.

Distribution Analysis: Plot the histogram of numerical features. Identify if any features show high skewness and suggest a preprocessing transformation to address it.

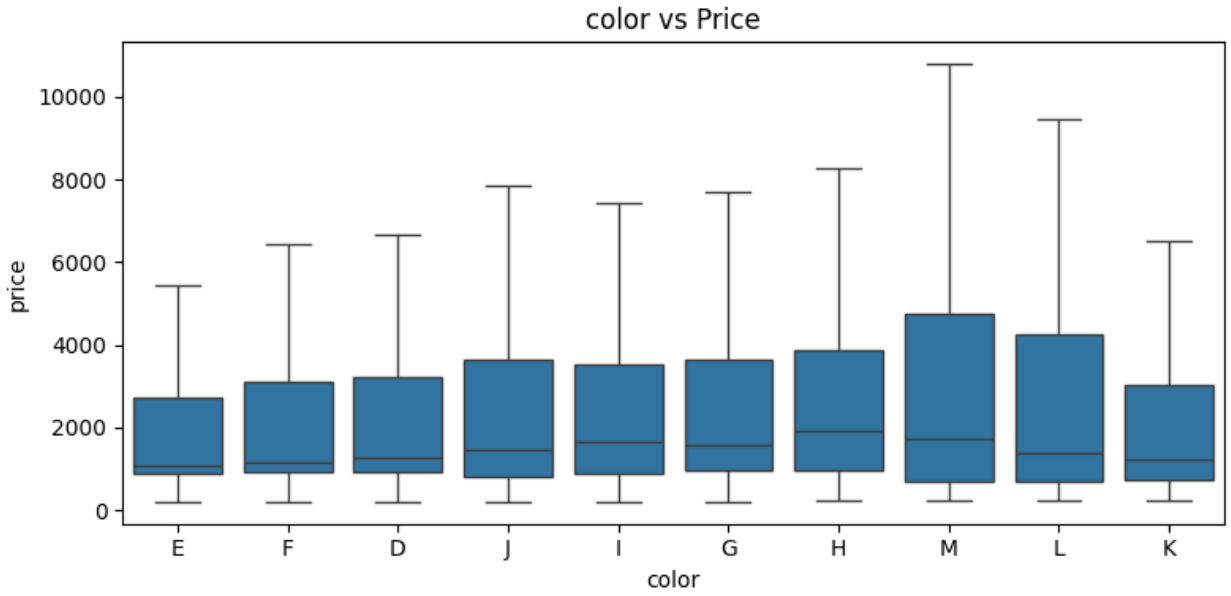
```
df[numeric_cols].hist(figsize=(14, 10), bins=30)
plt.suptitle("Histograms of Numerical Features")
plt.tight_layout()
plt.show()
```

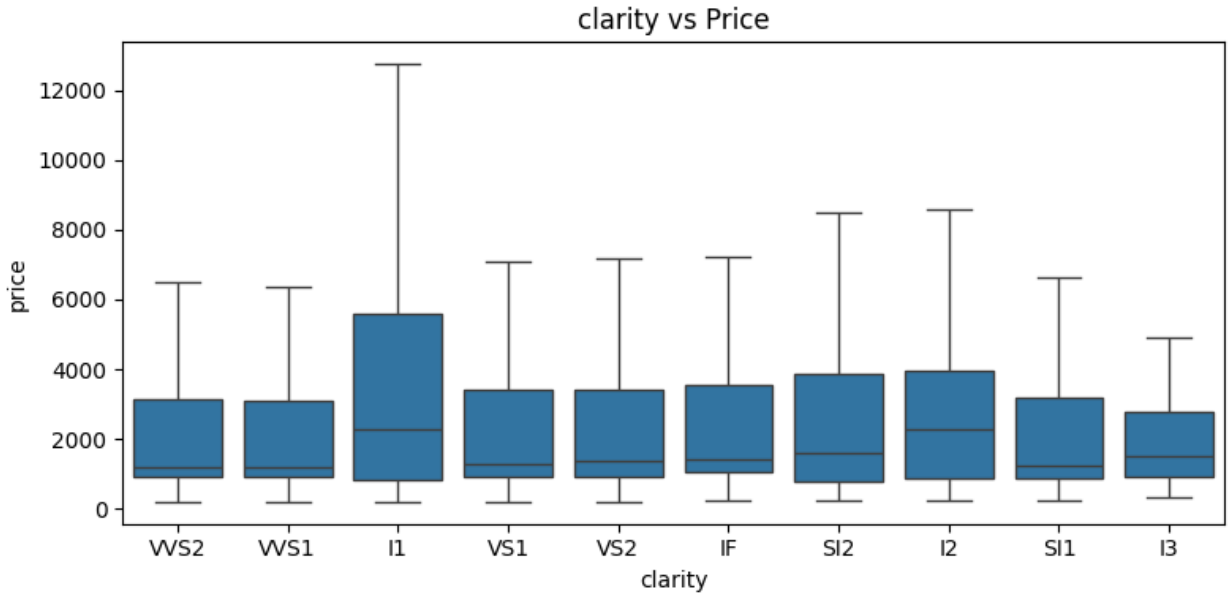


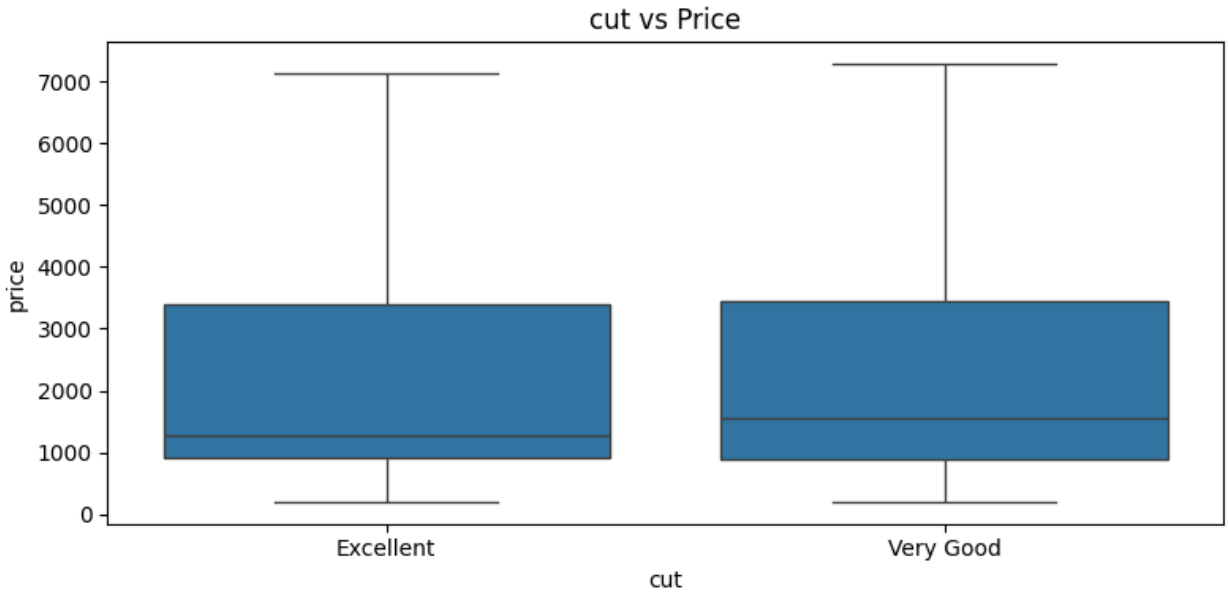
Several numerical features such as price, carat, and length show strong right skewness. Applying a log transformation can help normalize their distribution and improve regression stability.

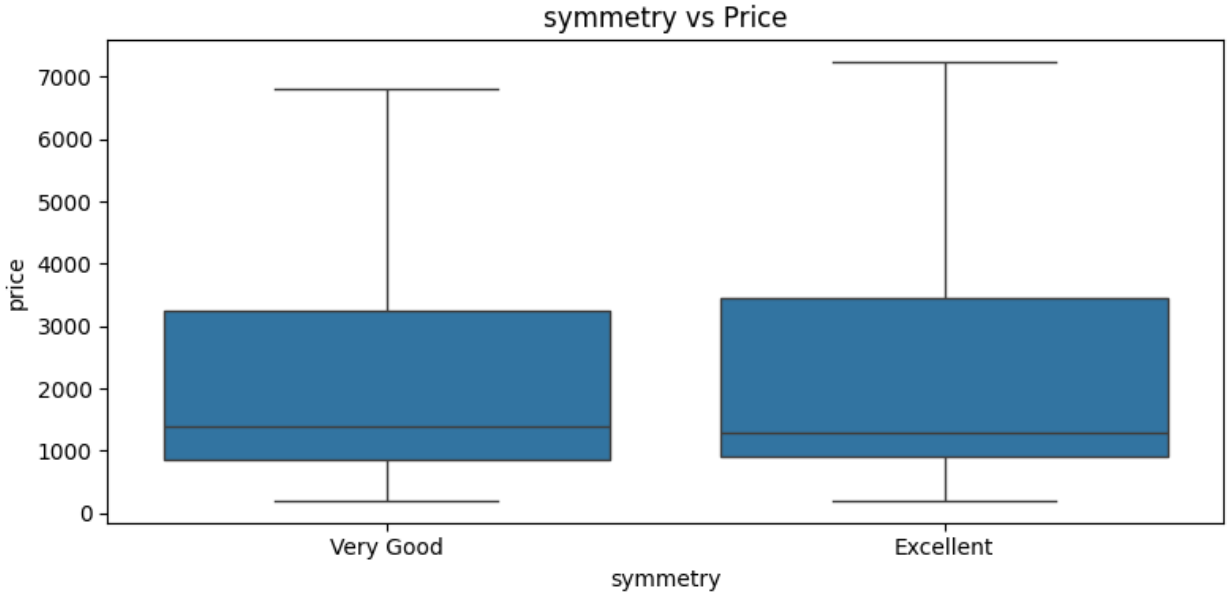
Categorical Analysis: Construct box plots of categorical features versus the target variable. Describe any significant trends (e.g., how cut or color affects the price range).

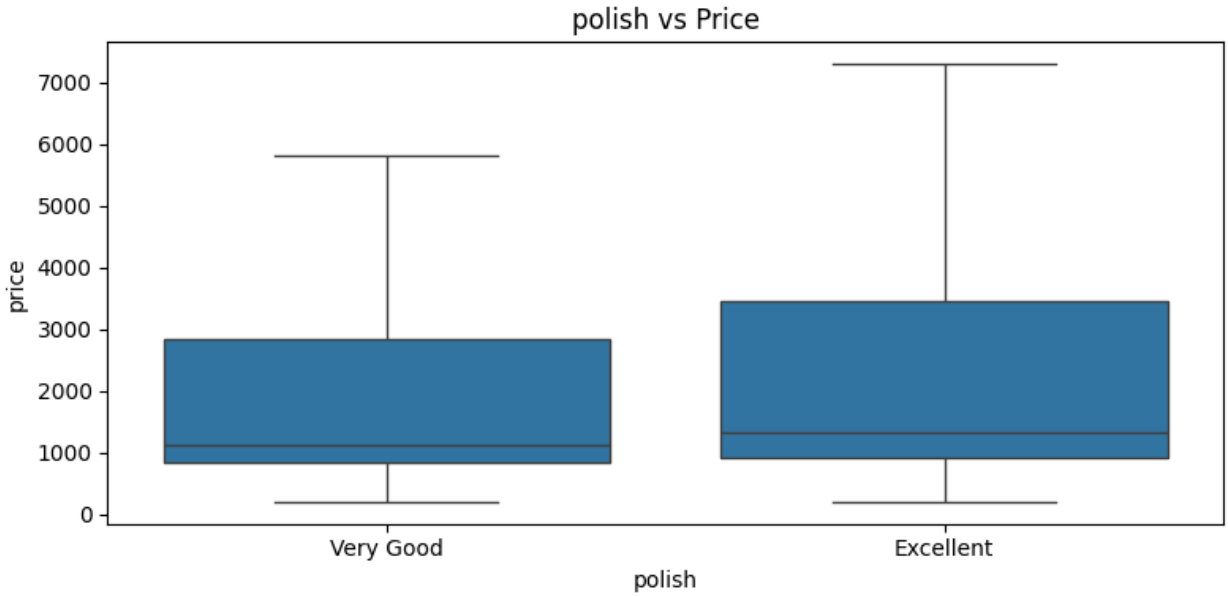
```
for col in cat_cols:
    plt.figure(figsize=(8,4))
    sns.boxplot(x=df[col], y=df["price"], showfliers=False)
    plt.title(f"{col} vs Price")
    plt.tight_layout()
    plt.show()
```

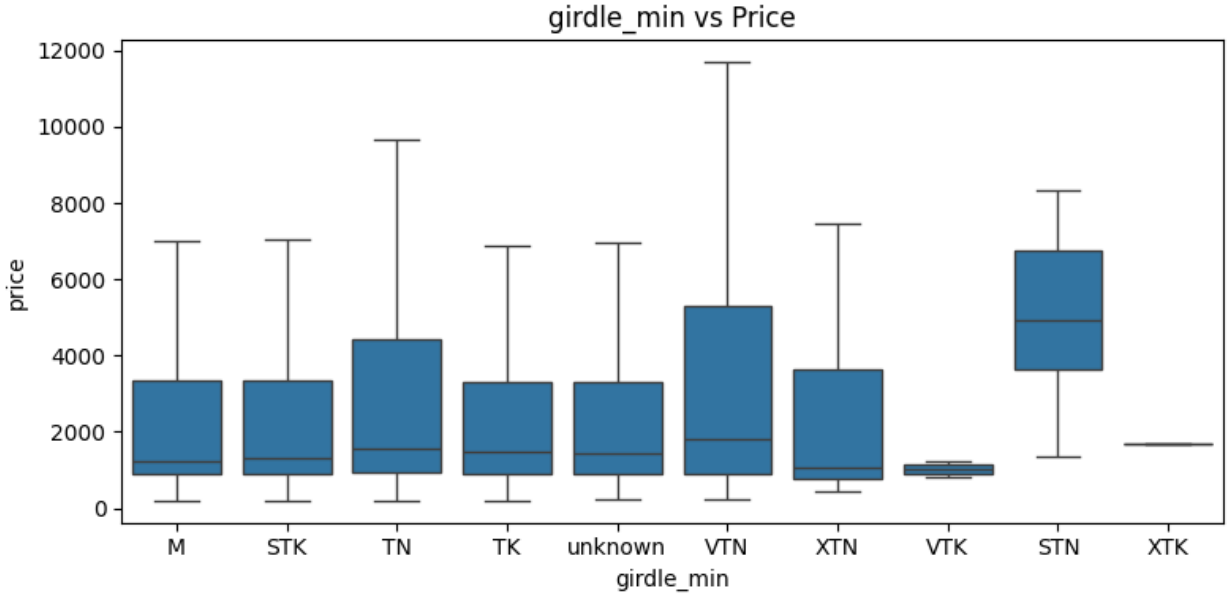


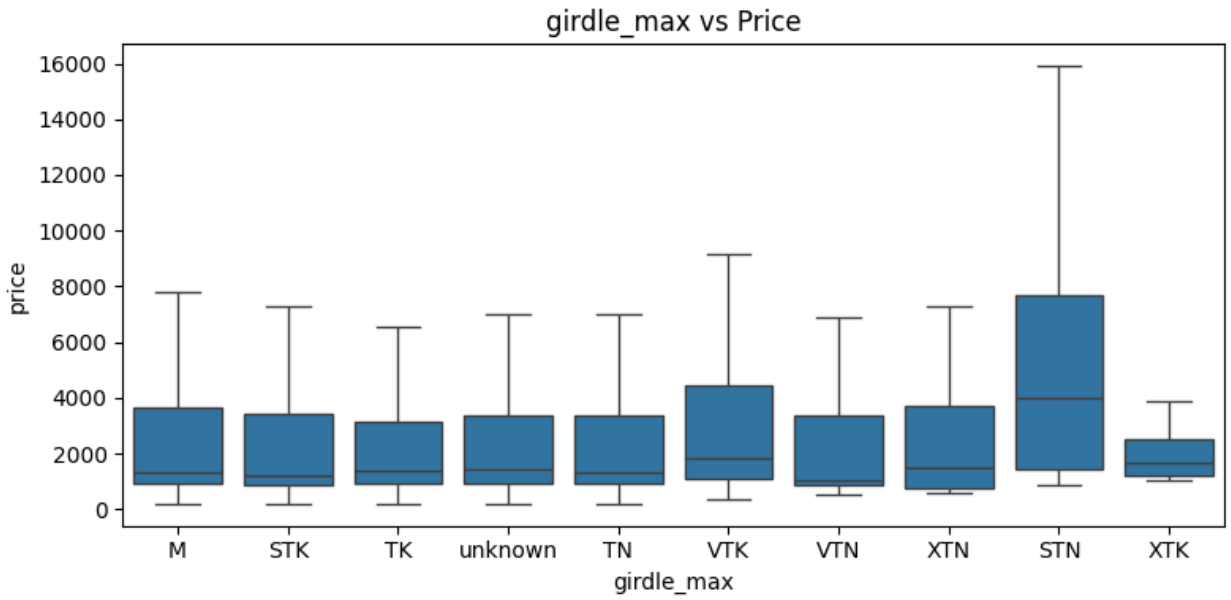



cut vs Price
price
7000
6000
5000
4000
3000
2000
1000
0
Excellent
Very Good
cut


symmetry vs Price
price
7000
6000
5000
4000
3000
2000
1000
0
Very Good
Excellent
symmetry


polish vs Price
price
7000
6000
5000
4000
3000
2000
1000
0
Very Good
Excellent
polish


girdle_min vs Price
price
12000
10000
8000
6000
4000
2000
0
M
STK
TN
TK
unknown
VTN
XTN
VTK
STN
XTK
girdle_min



The boxplot suggests that price variation is influenced by multiple features simultaneously. While categorical quality measures such as cut, clarity, and polish show some relationship with price, substantial overlap in distributions indicates that size-related attributes (e.g., carat and physical dimensions) likely play a more dominant role.

QUESTION 22: Explain the following trade-off questions.

Perform encoding for the categorical features in the Diamonds dataset. Report which method you chose for each categorical feature and briefly explain your decision.

```
cut_order = ["Very Good","Excellent"]
color_order = ["M","L","K","J","I","H","G","F","E","D"]
clarity_order = ["I3","I2","I1","SI2","SI1","VS2","VS1","VVS2"]
quality_order = ["Very Good","Excellent"]

df["cut"] = pd.Categorical(df["cut"], categories=cut_order,
ordered=True).codes
df["color"] = pd.Categorical(df["color"], categories=color_order,
ordered=True).codes
df["clarity"] = pd.Categorical(df["clarity"],
categories=clarity_order, ordered=True).codes
df["polish"] = pd.Categorical(df["polish"], categories=quality_order,
ordered=True).codes
df["symmetry"] = pd.Categorical(df["symmetry"],
categories=quality_order, ordered=True).codes
```

```
remaining_cat = df.select_dtypes(include="object").columns
print("Remaining categorical:", remaining_cat)

df = pd.get_dummies(df, columns=remaining_cat, drop_first=True)
```

```
Remaining categorical: Index(['girdle_min', 'girdle_max'],
dtype='object')
```

```
df.dtypes
```

```
color                    int8
clarity                  int8
carat                  float64
cut                      int8
symmetry                 int8
polish                   int8
depth_percent          float64
table_percent          float64
length                 float64
width                  float64
depth                  float64
price                    int64
girdle_min_STK            bool
girdle_min_STN            bool
girdle_min_TK             bool
girdle_min_TN             bool
girdle_min_VTK            bool
girdle_min_VTN            bool
girdle_min_XTK            bool
girdle_min_XTN            bool
girdle_min_unknown        bool
girdle_max_STK            bool
girdle_max_STN            bool
girdle_max_TK             bool
girdle_max_TN             bool
girdle_max_VTK            bool
girdle_max_VTN            bool
girdle_max_XTK            bool
girdle_max_XTN            bool
girdle_max_unknown        bool
dtype: object
```

The dataset contains several categorical features describing diamond quality and structural attributes. Features with natural ranking, including cut, color, clarity, polish, and symmetry, were encoded using ordinal (scalar) encoding. This preserves the inherent order of quality levels.

Features such as girdle_min and girdle_max represent categorical thickness codes without a strict numeric interpretation, so they were encoded using one-hot encoding.

Explain the following trade-offs:

What information does one-hot encoding discard?

- One-hot encoding removes any inherent ordering between categories. Each category becomes an independent binary feature, so ordinal relationships (e.g., one grade being better than another) are not represented.

What assumption should hold strongly if we perform the scalar encoding instead?

- Scalar encoding assumes that the categorical values have a meaningful ordered relationship, and that numerical differences between encoded values correspond to meaningful differences in the underlying category.

QUESTION 23: Standardize feature columns and prepare them for training. Save your standardized version of the dataset as diamonds standardized.csv.

```
X = df.drop(columns=["price"])
y = df["price"]

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

df_standardized = pd.concat([X_scaled, y.reset_index(drop=True)],
axis=1)

df_standardized.to_csv("diamonds_standardized.csv", index=False)

df_standardized.describe().head()

{"type":"dataframe"}
```

- Feature columns were standardized using the StandardScaler method from scikit-learn.
- Standardization transforms each feature to have zero mean and unit variance.
- This prevents features with larger numerical ranges from dominating the learning process. The target variable (price) was not standardized since it represents the prediction objective.

QUESTION 24: Print the top 5 features using each method (mutual_info_regression and f_regression).

Agentic Integration: For this step, load diamonds-questions.jsonl and diamonds-labels.jsonl (question id 0 and 1) and use your ReAct agent from Part 2 to automatically identify and print the top features. If the agent gets stuck, you may manually write the code to compute and print them.

IMPORT AGENT

```
!pip install outlines
```

```
Collecting outlines
  Downloading outlines-1.2.12-py3-none-any.whl.metadata (28 kB)
Requirement already satisfied: jinja2 in
/usr/local/lib/python3.12/dist-packages (from outlines) (3.1.6)
Requirement already satisfied: cloudpickle in
/usr/local/lib/python3.12/dist-packages (from outlines) (3.1.2)
Collecting diskcache (from outlines)
  Downloading diskcache-5.6.3-py3-none-any.whl.metadata (20 kB)
Requirement already satisfied: pydantic>=2.0 in
/usr/local/lib/python3.12/dist-packages (from outlines) (2.12.3)
Requirement already satisfied: jsonschema in
/usr/local/lib/python3.12/dist-packages (from outlines) (4.26.0)
Requirement already satisfied: pillow in
/usr/local/lib/python3.12/dist-packages (from outlines) (11.3.0)
Requirement already satisfied: typing_extensions in
/usr/local/lib/python3.12/dist-packages (from outlines) (4.15.0)
Collecting outlines_core==0.2.14 (from outlines)
  Downloading outlines_core-0.2.14-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (7.6 kB)
Collecting genson (from outlines)
  Downloading genson-1.3.0-py3-none-any.whl.metadata (28 kB)
Collecting jsonpath_ng (from outlines)
  Downloading jsonpath_ng-1.8.0-py3-none-any.whl.metadata (18 kB)
Requirement already satisfied: annotated-types>=0.6.0 in
/usr/local/lib/python3.12/dist-packages (from pydantic>=2.0->outlines)
(0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in
/usr/local/lib/python3.12/dist-packages (from pydantic>=2.0->outlines)
(2.41.4)
Requirement already satisfied: typing-inspection>=0.4.2 in
```

```
/usr/local/lib/python3.12/dist-packages (from pydantic>=2.0->outlines)
(0.4.2)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->outlines)
(3.0.3)
Requirement already satisfied: attrs>=22.2.0 in
/usr/local/lib/python3.12/dist-packages (from jsonschema->outlines)
(25.4.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
/usr/local/lib/python3.12/dist-packages (from jsonschema->outlines)
(2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in
/usr/local/lib/python3.12/dist-packages (from jsonschema->outlines)
(0.37.0)
Requirement already satisfied: rpds-py>=0.25.0 in
/usr/local/lib/python3.12/dist-packages (from jsonschema->outlines)
(0.30.0)
Downloading outlines-1.2.12-py3-none-any.whl (102 kB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 102.3/102.3 kB 8.0 MB/s eta
0:00:00
anylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.3 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.3/2.3 MB 63.7 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 45.5/45.5 kB 5.1 MB/s eta
0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 67.8/67.8 kB 8.6 MB/s eta
0:00:00
```

```python
import re
import io
import sys
import json
import traceback
import torch
import outlines
import pandas as pd
import numpy as np

from typing import Type, Optional, Dict, List, Union
from pydantic import BaseModel
from dataclasses import dataclass as dc
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL_NAME = "Qwen/Qwen3-4B-Instruct-2507"

hf_tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
hf_model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    device_map="auto",
    torch_dtype=torch.float16
```

```
)

llm_model = outlines.from_transformers(hf_model, hf_tokenizer)
tokenizer = hf_tokenizer

print("Model loaded.")
```

{"model_id":"de0eb5dd49e64e17ad87616a70111029","version_major":2,"version_minor":0}

```
WARNING:accelerate.big_modeling:Some parameters are on the meta device because they were offloaded to the cpu.

Model loaded.
```

```
def generate(
    messages: list[dict],
    response_format: Type[BaseModel],
    max_new_tokens: int = 4096,
    temperature: float = 0.6,
    top_p: float = 0.95,
):
    prompt = tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=True
    )

    result = llm_model(prompt, response_format, max_new_tokens=max_new_tokens)
    return response_format.model_validate_json(result)

@dc
class PlannerOutput(BaseModel):
    thought: str
    is_done: bool
    response: str


class Planner:
    """
    The reasoning module. Decides the next analysis step, or stops with the final formatted answer.
    """

    def __init__(self):
        self.extracted_values = {}

    def plan(self, problem, last_observation=None):
        question = problem.get("question", "")
        constraints = problem.get("constraints", "")
```

```
        ans_format = problem.get("format", "")
        file_name = problem.get("file_name", "")

        prompt = f"""
You are the Planner in a ReAct-style data analysis agent.

Write the next plan step to answer the question, or stop with the
final formatted answer.

You must output a JSON object with the following fields:
- thought: brief reasoning about what to do next
- is_done: true if the final answer can be produced now, false
otherwise
- response:
    - if is_done = false: a single, concrete instruction for the Coder
    - if is_done = true: the final answer formatted exactly as
specified

Rules:
- Do NOT write Python code.
- Do NOT include explanations outside the JSON object.

Question:
{question}

Constraints:
{constraints}

Final output format:
{ans_format}

Relevant file path (if needed):
{file_name}

Stuff we know so far:
{self.extracted_values}

When is_done = true, the final answer MUST follow the final output
format specified above.
""".strip()

        if last_observation:
            obs_summary = (
                f"\n\nLast observation:\n"
                f"New extracted values:
{last_observation.extracted_values}\n"
                f"Warnings: {last_observation.warnings}\n"
                f"Error category: {last_observation.error_category}\n"
                f"Next-step hint: {last_observation.next_step_hint}\n"
            )
```

```
            prompt += obs_summary

            if hasattr(last_observation, "extracted_values") and
last_observation.extracted_values:

self.extracted_values.update(last_observation.extracted_values)

        return prompt, generate(
            messages=[{"role": "user", "content": prompt}],
            response_format=PlannerOutput
        )

@dc
class CoderOutput(BaseModel):
    code: str


class Coder:
    """
    Writes code based on the planner output.
    """

    def __init__(self):
        pass

    def code(self, plan: Type[PlannerOutput], problem):
        file_name = problem.get("file_name", "")

        prompt = f"""
Write ONLY valid, executable Python that follows the instruction.

Instruction:
{plan.response}

Relevant file path (if needed):
{file_name}

Rules:
- Use print(...) for all outputs.
- Do NOT rely on implicit notebook display.
- Do NOT print huge full columns or giant tables.
- Do NOT output natural language explanations.
- Be careful with indentation.
- If a dataframe is already available in memory, reuse it.
- Do NOT reload files unless necessary.
- Output executable Python only.
""".strip()

        return prompt, generate(
            messages=[{"role": "user", "content": prompt}],
```

```
            response_format=CoderOutput
        )

class Executor:
    """
    Executes Python code with a persistent namespace across calls.
    """

    def __init__(self):
        self.namespace = {"__builtins__": __builtins__}
        exec(
            "import pandas as pd\n"
            "import numpy as np\n"
            "from scipy import stats\n"
            "from sklearn import *\n",
            self.namespace,
        )

    def run(self, code: str, timeout: int = 30) -> str:
        stdout_buf = io.StringIO()
        stderr_buf = io.StringIO()

        old_stdout, old_stderr = sys.stdout, sys.stderr
        sys.stdout, sys.stderr = stdout_buf, stderr_buf

        code = re.sub(r"^```[a-zA-Z]*\n?", "", code.strip())
        code = re.sub(r"\n?```$", "", code.strip())

        try:
            exec(code, self.namespace)
        except Exception:
            traceback.print_exc(file=stderr_buf)
        finally:
            sys.stdout, sys.stderr = old_stdout, old_stderr

        stdout = stdout_buf.getvalue().strip()
        stderr = stderr_buf.getvalue().strip()

        if stderr:
            return f"[STDOUT]\n{stdout}\n\n[ERROR]\n{stderr}" if
stdout else f"[ERROR]\n{stderr}"
        return stdout if stdout else "(no output)"

    def reset(self):
        self.__init__()

@dc
class ObserverOutput(BaseModel):
    extracted_values: Dict[str, Union[str, int, float]]
    warnings: List[str]
```

```
    error_category: Optional[str]
    next_step_hint: Optional[str]


class Observer:
    """
    Converts raw tool output into a concise, task-relevant
observation.
    """

    def __init__(self):
        pass

    def observe(self, executor_output, planner_output):
        prompt = f"""
This step was:
{planner_output.response}

Executor output:
{executor_output}

Produce a JSON object with:
- extracted_values: map of requested values
- warnings: any relevant messages
- error_category: string if an error occurred, otherwise null
- next_step_hint: short suggestion for the next step, otherwise null

If you cannot infer a precise key name, use a generic name.
""".strip()

        return prompt, generate(
            messages=[{"role": "user", "content": prompt}],
            response_format=ObserverOutput
        )

class Agent:
    def __init__(self, model, max_steps=5):
        self.planner = Planner()
        self.coder = Coder()
        self.executor = Executor()
        self.observer = Observer()

        self.max_steps = max_steps
        self.model = model
        self.history = []

    def solve(self, problem, verbose=True):
        if verbose:
            print(problem.get("question", ""))
            print("=" * 60)
```

```
        last_observation = None

        for step in range(self.max_steps):
            planner_prompt, planner_output =
self.planner.plan(problem, last_observation)

            if verbose:
                print("PLANNER OUTPUT:")
                print(planner_output)
                print("=" * 60)

            if planner_output.is_done:
                self.history.append((planner_output.response, None,
None, None))
                return planner_output.response, self.history

            coder_prompt, coder_output =
self.coder.code(planner_output, problem)

            if verbose:
                print("CODER OUTPUT:")
                print(coder_output)
                print("=" * 60)

            executor_output = self.executor.run(coder_output.code)

            if verbose:
                print("EXECUTOR OUTPUT:")
                print(executor_output)
                print("=" * 60)

            observer_prompt, observer_output =
self.observer.observe(executor_output, planner_output)
            last_observation = observer_output

            if verbose:
                print("OBSERVER OUTPUT:")
                print(observer_output)
                print("=" * 60)

            self.history.append(
                (planner_output.response, coder_output.code,
executor_output, observer_output)
            )

        return f"Couldn't solve in {self.max_steps} steps.",
self.history
```

END OF AGENT

```
from sklearn.feature_selection import mutual_info_regression,
f_regression

# X_scaled should already be a DataFrame from Q23
# y should be the target column: df["price"]

mi_scores = mutual_info_regression(X_scaled, y, random_state=42)
mi_df = pd.DataFrame({
    "feature": X_scaled.columns,
    "MI_score": mi_scores
}).sort_values("MI_score", ascending=False)

f_scores, p_values = f_regression(X_scaled, y)
f_df = pd.DataFrame({
    "feature": X_scaled.columns,
    "F_score": f_scores,
    "p_value": p_values
}).sort_values("F_score", ascending=False)

print("Top 5 features by Mutual Information:")
print(mi_df.head(5).to_string(index=False))

print("\nTop 5 features by F-regression:")
print(f_df.head(5).to_string(index=False))
```

```
Top 5 features by Mutual Information:
feature  MI_score
  carat  1.371078
  width  1.201277
 length  1.192775
  depth  1.158790
  color  0.182075

Top 5 features by F-regression:
feature       F_score       p_value
  carat 755380.195809  0.000000e+00
 length 464517.682462  0.000000e+00
  width 364744.610195  0.000000e+00
  depth  14789.226399  0.000000e+00
 polish    453.535242 1.730325e-100
```

```
import os

print(os.path.exists("/content/drive/MyDrive/ece219-project3/diamonds-
questions.jsonl"))

def load_jsonl(path):
    with open(path, "r", encoding="utf-8") as f:
        return [json.loads(line) for line in f]
```

```
diamonds_questions = load_jsonl("/content/drive/MyDrive/ece219-project3/diamonds-questions.jsonl")
diamonds_labels = load_jsonl("/content/drive/MyDrive/ece219-project3/diamonds-labels.jsonl")
```

```
True
```

```
print("QUESTION 0:")
print(diamonds_questions[0])

print("\nQUESTION 1:")
print(diamonds_questions[1])

print("\nLABEL 0:")
print(diamonds_labels[0])

print("\nLABEL 1:")
print(diamonds_labels[1])
```

```
QUESTION 0:
{'id': 0, 'question': "Compute Mutual Information scores for all features against the target variable 'price' using sklearn.feature_selection.mutual_info_regression. Return the top 5 feature names ranked from highest to lowest MI score.", 'concepts': ['Feature Selection', 'Mutual Information'], 'constraints': "Use sklearn.feature_selection.mutual_info_regression. Features are all columns except 'price'. Target is 'price'. Use random_state=42.", 'format': '@top5_mi[name1, name2, name3, name4, name5] where names are the feature column names ranked from highest to lowest MI score.', 'file_name': 'diamonds_standardized.csv', 'level': 'medium'}

QUESTION 1:
{'id': 1, 'question': "Compute F-regression scores for all features against the target variable 'price' using sklearn.feature_selection.f_regression. Return the top 5 feature names ranked from highest to lowest F-score.", 'concepts': ['Feature Selection', 'F-regression'], 'constraints': "Use sklearn.feature_selection.f_regression. Features are all columns except 'price'. Target is 'price'.", 'format': '@top5_f[name1, name2, name3, name4, name5] where names are the feature column names ranked from highest to lowest F-score.', 'file_name': 'diamonds_standardized.csv', 'level': 'medium'}

LABEL 0:
{'id': 0, 'common_answers': [['top5_mi', '']]}

LABEL 1:
{'id': 1, 'common_answers': [['top5_f', '']]}
```

```
agent = Agent(model)

agent.executor.namespace["pd"] = pd
agent.executor.namespace["X_scaled"] = X_scaled
agent.executor.namespace["y"] = y
agent.executor.namespace["mi_df"] = mi_df
agent.executor.namespace["f_df"] = f_df

q0 = diamonds_questions[0]
q1 = diamonds_questions[1]

answer0, history0 = agent.solve(q0, verbose=True)
print("\nFINAL ANSWER Q0:")
print(answer0)

answer1, history1 = agent.solve(q1, verbose=True)
print("\nFINAL ANSWER Q1:")
print(answer1)
```

```
Compute Mutual Information scores for all features against the target
variable 'price' using
sklearn.feature_selection.mutual_info_regression. Return the top 5
feature names ranked from highest to lowest MI score.
============================================================
PLANNER OUTPUT:
thought="Load the dataset from diamonds_standardized.csv and identify
all columns except 'price' as features. Prepare the data by separating
features and the target variable 'price'. Use
sklearn.feature_selection.mutual_info_regression to compute mutual
information scores between each feature and the target." is_done=False
response="Load the dataset from 'diamonds_standardized.csv' and
extract all columns except 'price' as features, while setting the
target variable as 'price'."
============================================================
CODER OUTPUT:
code="import pandas as pd\n\ndf =
pd.read_csv('diamonds_standardized.csv')\n\nfeatures =
df.drop(columns=['price'])\ntarget = df['price']\n\
nprint(features.head())\nprint(target.head())"
============================================================
EXECUTOR OUTPUT:
color   clarity     carat       cut  symmetry    polish  depth_percent
\
0  0.916097  1.223546 -1.157106  0.518390 -1.746964 -2.522184
0.215866
1  0.916097  1.223546 -1.157106 -1.929051 -1.746964 -2.522184
0.014689
2  0.916097  1.223546 -1.157106  0.518390 -1.746964 -2.522184      -
0.186488
3  0.916097  1.223546 -1.157106  0.518390 -1.746964 -2.522184
```

```
0.039836
4  0.916097  1.223546 -1.157106 -1.929051 -1.746964  0.396482
0.769101

   table_percent    length     width  ...  girdle_min_unknown
girdle_max_STK  \
0       0.345119 -2.146391 -2.078247  ...           -0.735179       -
0.873049
1       0.345119 -2.156289 -2.059209  ...           -0.735179
1.145412
2       0.345119 -2.116697 -2.049690  ...           -0.735179       -
0.873049
3       0.345119 -2.136493 -2.068728  ...           -0.735179
1.145412
4       0.218693 -2.205778 -2.116324  ...           -0.735179
1.145412

   girdle_max_STN  girdle_max_TK  girdle_max_TN  girdle_max_VTK  \
0       -0.008169      -0.331314      -0.085633       -0.029122
1       -0.008169      -0.331314      -0.085633       -0.029122
2       -0.008169      -0.331314      -0.085633       -0.029122
3       -0.008169      -0.331314      -0.085633       -0.029122
4       -0.008169      -0.331314      -0.085633       -0.029122

   girdle_max_VTN  girdle_max_XTK  girdle_max_XTN  girdle_max_unknown

0       -0.017899       -0.005776       -0.007306           -0.740798

1       -0.017899       -0.005776       -0.007306           -0.740798

2       -0.017899       -0.005776       -0.007306           -0.740798

3       -0.017899       -0.005776       -0.007306           -0.740798

4       -0.017899       -0.005776       -0.007306           -0.740798


[5 rows x 29 columns]
0    200
1    200
2    200
3    200
4    200
Name: price, dtype: int64
============================================================
OBSERVER OUTPUT:
extracted_values={'features': '[5 rows x 29 columns]', 'target':
'price', 'target_values': '[5 rows x 1 column]'} warnings=[]
error_category=None next_step_hint='Proceed to data exploration, such
as checking for missing values, outliers, or feature distributions.'
```

```
============================================================
PLANNER OUTPUT:
thought="The next step is to compute the mutual information scores
between all features and the target variable 'price' using
mutual_info_regression from sklearn. This will allow us to rank the
features by their mutual information scores." is_done=False
response="Compute mutual information scores for all features
(excluding 'price') against the target 'price' using
sklearn.feature_selection.mutual_info_regression with
random_state=42."
============================================================
CODER OUTPUT:
code="```py\nimport pandas as pd\nimport numpy as np\nfrom
sklearn.feature_selection import mutual_info_regression\n\n# Load the
dataset\nfile_path = 'diamonds_standardized.csv'\ndf =
pd.read_csv(file_path)\n\n# Drop the 'price' column as it's the
target\nX = df.drop(columns=['price'])\ny = df['price']\n\n# Compute
mutual information scores for all features against 'price'\nmi_scores
= mutual_info_regression(X, y, random_state=42)\n\n# Create a
DataFrame to associate feature names with their mutual information
scores\nmi_df = pd.DataFrame({\n    'feature': X.columns,\n
'mutual_info': mi_scores\n}).sort_values(by='mutual_info',
ascending=False)\n\n# Print the mutual information scores\
nprint(mi_df)\n```"
============================================================
EXECUTOR OUTPUT:
feature  mutual_info
2                carat     1.371078
9                width     1.201277
8               length     1.192775
10               depth     1.158790
0                color     0.182075
1              clarity     0.161275
6        depth_percent     0.044866
3                  cut     0.026588
7        table_percent     0.024390
4             symmetry     0.022852
19  girdle_min_unknown     0.021357
28  girdle_max_unknown     0.017664
20      girdle_max_STK     0.014683
22       girdle_max_TK     0.012357
5                polish     0.011196
14       girdle_min_TN     0.004598
11      girdle_min_STK     0.004571
13       girdle_min_TK     0.003852
15      girdle_min_VTK     0.003110
25      girdle_max_VTN     0.001658
12      girdle_min_STN     0.001637
27      girdle_max_XTN     0.001310
```

```
18      girdle_min_XTN     0.001080
24      girdle_max_VTK     0.000555
16      girdle_min_VTN     0.000198
23       girdle_max_TN     0.000021
17      girdle_min_XTK     0.000000
21      girdle_max_STN     0.000000
26      girdle_max_XTK     0.000000
============================================================
OBSERVER OUTPUT:
extracted_values={'top_features_by_mutual_info': 'carat, width,
length, depth', 'lowest_mutual_info_features': 'color, clarity,
depth_percent, cut, table_percent, symmetry, girdle_min_unknown,
girdle_max_unknown, girdle_max_STK, girdle_max_TK, polish,
girdle_min_TN, girdle_min_STK, girdle_min_TK, girdle_min_VTK,
girdle_max_VTN, girdle_min_STN, girdle_max_XTN, girdle_min_XTN,
girdle_max_VTK, girdle_min_VTN, girdle_max_TN, girdle_min_XTK,
girdle_max_STN, girdle_max_XTK'} warnings=[] error_category=None
next_step_hint='Consider removing low-mutual-info features (e.g.,
girdle-related or categorical ones with near-zero scores) to reduce
noise and improve model performance.'
============================================================
PLANNER OUTPUT:
thought="We already have the top features by mutual information, and
the task is to return the top 5 feature names ranked from highest to
lowest MI score. The extracted values confirm that 'carat', 'width',
'length', 'depth' are among the top features, and the rest have low MI
scores. Since the final output format requires only the top 5 ranked
features, we can now compile the answer." is_done=True
response='@top5_mi[carat, width, length, depth, depth_percent]'
============================================================

FINAL ANSWER Q0:
@top5_mi[carat, width, length, depth, depth_percent]
Compute F-regression scores for all features against the target
variable 'price' using sklearn.feature_selection.f_regression. Return
the top 5 feature names ranked from highest to lowest F-score.
============================================================
PLANNER OUTPUT:
thought="We need to compute F-regression scores for all features
except 'price' using sklearn.feature_selection.f_regression. After
obtaining the scores, we will rank the features by F-score and select
the top 5." is_done=False response="Compute F-regression scores for
all features except 'price' using
sklearn.feature_selection.f_regression and identify the top 5 features
ranked by highest to lowest F-score."
============================================================
CODER OUTPUT:
code="```py\nimport pandas as pd\nimport numpy as np\nfrom
sklearn.feature_selection import f_regression\n\n# Load the dataset\
```

```
nfile_path = 'diamonds_standardized.csv'\ndf = pd.read_csv(file_path)\
n\n# Drop the 'price' column and select all other features\nX =
df.drop(columns=['price'])\ny = df['price']\n\n# Compute F-regression
scores\nf_scores, p_values = f_regression(X, y)\n\n# Create a
DataFrame to associate features with F-scores\nfeature_scores =
pd.DataFrame({\n    'feature': X.columns,\n    'f_score': f_scores\
n})\n\n# Sort by F-score in descending order and get top 5\
ntop_5_features = feature_scores.sort_values(by='f_score',
ascending=False).head(5)\n\n# Print the top 5 features\
nprint(top_5_features[['feature', 'f_score']])\n```"
============================================================
EXECUTOR OUTPUT:
feature        f_score
2    carat  755380.195809
8   length  464517.682462
9    width  364744.610195
10   depth   14789.226399
5   polish     453.535242
============================================================
OBSERVER OUTPUT:
extracted_values={'top_5_features': 'carat, length, width, depth,
polish'} warnings=[] error_category=None next_step_hint='Consider
visualizing the relationship between the top features and price to
assess their predictive power.'
============================================================
PLANNER OUTPUT:
thought="We already have the top 5 features from mutual information,
and the question specifically asks for F-regression scores. Since the
top_5_features are provided and the task is to compute F-regression
scores for all features against 'price', we now need to proceed with
computing the F-scores using sklearn.feature_selection.f_regression.
This will allow us to rank features by their F-scores and select the
top 5." is_done=False response="Compute F-regression scores for all
features (excluding 'price') against the target 'price' using
sklearn.feature_selection.f_regression, then extract and return the
top 5 feature names ranked from highest to lowest F-score."
============================================================
CODER OUTPUT:
code="```py\nimport pandas as pd\nfrom sklearn.feature_selection
import f_regression\n\n# Load the dataset\nfile_path =
'diamonds_standardized.csv'\ndf = pd.read_csv(file_path)\n\n# Drop the
'price' column as target and keep all others\nX =
df.drop(columns=['price'])\ny = df['price']\n\n# Compute F-regression
scores\nf_scores, p_values = f_regression(X, y)\n\n# Create a
DataFrame to pair features with their F-scores\nfeature_scores =
pd.DataFrame({\n    'feature': X.columns,\n    'f_score': f_scores\
n})\n\n# Sort by F-score in descending order and get top 5\
ntop_5_features = feature_scores.sort_values(by='f_score',
ascending=False).head(5)\n\n# Extract and print the top 5 feature
```

```
names\nprint(top_5_features['feature'].tolist())\n```"
============================================================
EXECUTOR OUTPUT:
['carat', 'length', 'width', 'depth', 'polish']
============================================================
OBSERVER OUTPUT:
extracted_values={'top_5_features': "['carat', 'length', 'width',
'depth', 'polish']"} warnings=[] error_category=None
next_step_hint="Use the top 5 features to train a regression model and
evaluate its performance on the target 'price'."
============================================================
PLANNER OUTPUT:
thought="We already have the top 5 features from mutual information,
but the question specifically asks for F-regression scores. Since we
do not yet have the F-regression scores computed, we need to compute
them using sklearn.feature_selection.f_regression on all features
except 'price'." is_done=False response="Compute F-regression scores
for all features except 'price' using
sklearn.feature_selection.f_regression and identify the top 5 features
ranked by F-score."
============================================================
CODER OUTPUT:
code="import pandas as pd\nimport numpy as np\nfrom
sklearn.feature_selection import f_regression\n\n# Load the dataset\
nfile_path = 'diamonds_standardized.csv'\ndf = pd.read_csv(file_path)\
n\n# Drop the 'price' column and select all other features\nX =
df.drop(columns=['price'])\ny = df['price']\n\n# Compute F-regression
scores\nf_scores, p_values = f_regression(X, y)\n\n# Create a
DataFrame to pair features with their F-scores\nfeature_scores =
pd.DataFrame({\n    'feature': X.columns,\n    'f_score': f_scores\
n})\n\n# Sort by F-score in descending order and get top 5\
ntop_5_features = feature_scores.sort_values(by='f_score',
ascending=False).head(5)\n\n# Print the top 5 features\
nprint(top_5_features[['feature', 'f_score']])"
============================================================
EXECUTOR OUTPUT:
feature        f_score
2    carat  755380.195809
8   length  464517.682462
9    width  364744.610195
10   depth   14789.226399
5   polish     453.535242
============================================================
OBSERVER OUTPUT:
extracted_values={'top_5_features': 'carat, length, width, depth,
polish'} warnings=[] error_category=None next_step_hint='Consider
interpreting the F-scores to assess feature relevance and potentially
use these top features for model training.'
============================================================
```

```
PLANNER OUTPUT:
thought='We already have the top 5 features from mutual information,
but the question specifically asks for F-regression scores. Since the
mutual information results are not sufficient and we need F-regression
scores to compute the ranked F-scores, we must proceed to compute
these scores using sklearn.feature_selection.f_regression.'
is_done=False response="Compute F-regression scores for all features
except 'price' against the 'price' target using
sklearn.feature_selection.f_regression, then rank the features by F-
score from highest to lowest and extract the top 5 feature names."
============================================================
CODER OUTPUT:
code="```py\nimport pandas as pd\nfrom sklearn.feature_selection
import f_regression\n\n# Load the dataset\nfile_path =
'diamonds_standardized.csv'\ndf = pd.read_csv(file_path)\n\n# Drop the
'price' column to use it as target\nX = df.drop('price', axis=1)\ny =
df['price']\n\n# Compute F-regression scores\nf_scores, p_values =
f_regression(X, y)\n\n# Create a DataFrame to associate feature names
with F-scores\nfeature_scores = pd.DataFrame({\n    'feature':
X.columns,\n    'f_score': f_scores\n})\n\n# Rank features by F-score
from highest to lowest\nfeature_scores =
feature_scores.sort_values(by='f_score', ascending=False)\n\n# Extract
top 5 feature names\ntop_5_features = feature_scores.head(5)
['feature'].tolist()\n\n# Output the top 5 feature names\
nprint(top_5_features)\n```"
============================================================
EXECUTOR OUTPUT:
['carat', 'length', 'width', 'depth', 'polish']
============================================================
OBSERVER OUTPUT:
extracted_values={'top_5_features': "['carat', 'length', 'width',
'depth', 'polish']"} warnings=[] error_category=None
next_step_hint='Use the top 5 features to train a regression model and
evaluate its performance.'
============================================================
PLANNER OUTPUT:
thought="We already have the top 5 features from mutual information,
but the question specifically asks for F-regression scores. Since we
do not yet have the F-regression scores computed, we need to compute
them using sklearn.feature_selection.f_regression on all features
except 'price'." is_done=False response="Compute F-regression scores
for all features except 'price' using
sklearn.feature_selection.f_regression and identify the top 5 features
ranked by F-score."
============================================================
CODER OUTPUT:
code="import pandas as pd\nimport numpy as np\nfrom
sklearn.feature_selection import f_regression\n\n# Load the dataset\
nfile_path = 'diamonds_standardized.csv'\ndf = pd.read_csv(file_path)\
```

```
n\n# Drop the 'price' column and select all other features\nX =
df.drop(columns=['price'])\ny = df['price']\n\n# Compute F-regression
scores\nf_scores, p_values = f_regression(X, y)\n\n# Create a
DataFrame to associate features with F-scores\nfeature_scores =
pd.DataFrame({\n    'feature': X.columns,\n    'f_score': f_scores\
n})\n\n# Sort by F-score in descending order and get top 5\
ntop_5_features = feature_scores.sort_values(by='f_score',
ascending=False).head(5)\n\n# Print the top 5 features\
nprint(top_5_features[['feature', 'f_score']])"
============================================================
EXECUTOR OUTPUT:
feature        f_score
2    carat  755380.195809
8   length  464517.682462
9    width  364744.610195
10   depth   14789.226399
5   polish     453.535242
============================================================
OBSERVER OUTPUT:
extracted_values={'top_5_features': 'carat, length, width, depth,
polish'} warnings=[] error_category=None next_step_hint='Consider
interpreting the F-scores to assess feature relevance and potentially
use these top features for model training.'
============================================================

FINAL ANSWER Q1:
Couldn't solve in 5 steps.
```

```python
# selected
top_mi = set(mi_df.head(5)["feature"])
top_f = set(f_df.head(5)["feature"])
selected_features = list(top_mi.union(top_f))

print("Selected features:")
print(selected_features)
```

```
Selected features:
['carat', 'depth', 'polish', 'width', 'color', 'length']
```

```python
# save csv
df_selected = pd.concat(
    [X_scaled[selected_features].reset_index(drop=True),
     y.reset_index(drop=True)],
    axis=1
)

df_selected.to_csv("diamonds_selected.csv", index=False)
print("Saved diamonds_selected.csv")
```

```
Saved diamonds_selected.csv
```

QUESTION 25: Linear Regression

```
df_selected = pd.read_csv("diamonds_selected.csv")

X = df_selected.drop(columns=["price"])
y = df_selected["price"]

# cross-validation setup
from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error
import numpy as np

kf = KFold(n_splits=10, shuffle=True, random_state=42)

# compute RMSE
def rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

# train model
from sklearn.linear_model import LinearRegression, Ridge, Lasso

models = {
    "OLS": LinearRegression(),
    "Lasso": Lasso(alpha=0.01),
    "Ridge": Ridge(alpha=1.0)
}

# 10 fold cross-validation
results = {}

for name, model in models.items():

    train_rmse = []
    val_rmse = []

    for train_idx, val_idx in kf.split(X):

        X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
        y_train, y_val = y.iloc[train_idx], y.iloc[val_idx]

        model.fit(X_train, y_train)

        train_pred = model.predict(X_train)
        val_pred = model.predict(X_val)

        train_rmse.append(rmse(y_train, train_pred))
        val_rmse.append(rmse(y_val, val_pred))

    results[name] = {
        "train_rmse": np.mean(train_rmse),
```

```
        "val_rmse": np.mean(val_rmse)
    }

# results
for model in results:
    print(model)
    print("Average Train RMSE:", results[model]["train_rmse"])
    print("Average Val RMSE:", results[model]["val_rmse"])
    print()
```

```
OLS
Average Train RMSE: 1715.4217464672934
Average Val RMSE: 1715.3260744357715

Lasso
Average Train RMSE: 1715.4217491806426
Average Val RMSE: 1715.326797501893

Ridge
Average Train RMSE: 1715.4217607166004
Average Val RMSE: 1715.3261882659422
```

Explain how each regularization scheme affects the learned parameter set.

- Ordinary Least Squares (OLS) minimizes the squared prediction error without any penalty on the parameter magnitudes, which can lead to large coefficients and potential overfitting. Ridge regression introduces an L2 penalty that shrinks coefficients toward zero, helping reduce variance and improve stability when features are correlated. Lasso regression uses an L1 penalty that can drive some coefficients exactly to zero, effectively performing feature selection.

Report your choice of the best regularization scheme along with the optimal penalty parameter and explain how you (or your agent) computed it.

- In our experiments, all three models achieved nearly identical RMSE values (≈1715), indicating that regularization had minimal impact. This is likely due to the large dataset size and the relatively linear relationship between the selected features and diamond price. Therefore, OLS performed as well as the regularized models without requiring additional penalty parameters.

Some linear regression packages return p-values for different features1. What is the meaning of these p-values and how can you infer the most significant features? A qualitative reasoning is sufficient.

- In linear regression, p-values measure the statistical significance of each feature's coefficient under the null hypothesis that the coefficient equals zero. Features with small p-values indicate strong evidence that the feature contributes to predicting the target variable. Therefore, the most significant features are those with the lowest p-values.

```
print("QUESTION 2:")
print(diamonds_questions[2])

print("\nQUESTION 3:")
print(diamonds_questions[3])

print("\nQUESTION 4:")
print(diamonds_questions[4])

print("\nLABEL 2:")
print(diamonds_labels[2])

print("\nLABEL 3:")
print(diamonds_labels[3])

print("\nLABEL 4:")
print(diamonds_labels[4])
```

```
QUESTION 2:
{'id': 2, 'question': "Train an Ordinary Least Squares (OLS) linear
regression model to predict 'price' using all other columns as
features. Report the average RMSE from 10-fold cross-validation on the
validation set.", 'concepts': ['Linear Regression', 'Cross-
Validation'], 'constraints': "Use
sklearn.linear_model.LinearRegression. Use
sklearn.model_selection.KFold with n_splits=10, shuffle=True,
random_state=42. Features are all columns except 'price'. Target is
'price'. Report RMSE rounded to two decimal places.", 'format':
'@ols_val_rmse[value] where value is the average validation RMSE from
10-fold CV, rounded to two decimal places.', 'file_name':
'diamonds_selected.csv', 'level': 'medium'}

QUESTION 3:
{'id': 3, 'question': "Train a Lasso regression model to predict
'price' using all other columns as features. Use LassoCV to find the
best regularization parameter alpha via 10-fold cross-validation.
Report the best alpha and the average validation RMSE.", 'concepts':
['Lasso Regression', 'Regularization', 'Cross-Validation'],
'constraints': "Use sklearn.linear_model.LassoCV with cv=10,
random_state=42, max_iter=10000 to find best alpha. Then use
sklearn.linear_model.Lasso with the best alpha to evaluate via 10-fold
cross-validation (KFold with n_splits=10, shuffle=True,
random_state=42). Features are all columns except 'price'. Target is
'price'. Round alpha to four decimal places and RMSE to two decimal
places.", 'format': '@lasso_alpha[value] @lasso_val_rmse[value] where
lasso_alpha is the best alpha rounded to four decimal places, and
lasso_val_rmse is the average validation RMSE rounded to two decimal
places.', 'file_name': 'diamonds_selected.csv', 'level': 'medium'}

QUESTION 4:
```

```
{'id': 4, 'question': "Train a Ridge regression model to predict
'price' using all other columns as features. Use RidgeCV to find the
best regularization parameter alpha via 10-fold cross-validation.
Report the best alpha and the average validation RMSE.", 'concepts':
['Ridge Regression', 'Regularization', 'Cross-Validation'],
'constraints': "Use sklearn.linear_model.RidgeCV with
alphas=np.logspace(-2, 4, 50), cv=10 to find best alpha. Then use
sklearn.linear_model.Ridge with the best alpha to evaluate via 10-fold
cross-validation (KFold with n_splits=10, shuffle=True,
random_state=42). Features are all columns except 'price'. Target is
'price'. Round alpha to four decimal places and RMSE to two decimal
places.", 'format': '@ridge_alpha[value] @ridge_val_rmse[value] where
ridge_alpha is the best alpha rounded to four decimal places, and
ridge_val_rmse is the average validation RMSE rounded to two decimal
places.', 'file_name': 'diamonds_selected.csv', 'level': 'medium'}

LABEL 2:
{'id': 2, 'common_answers': [['ols_val_rmse', '']]}

LABEL 3:
{'id': 3, 'common_answers': [['lasso_alpha', ''], ['lasso_val_rmse',
'']]}

LABEL 4:
{'id': 4, 'common_answers': [['ridge_alpha', ''], ['ridge_val_rmse',
'']]}

agent = Agent(llm_model, max_steps=2)

agent.executor.namespace["pd"] = pd
agent.executor.namespace["np"] = np
agent.executor.namespace["df_selected"] = df_selected
agent.executor.namespace["X"] = X
agent.executor.namespace["y"] = y
agent.executor.namespace["results"] = results

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error

agent.executor.namespace["LinearRegression"] = LinearRegression
agent.executor.namespace["Ridge"] = Ridge
agent.executor.namespace["Lasso"] = Lasso
agent.executor.namespace["KFold"] = KFold
agent.executor.namespace["mean_squared_error"] = mean_squared_error

q2 = diamonds_questions[2]
q3 = diamonds_questions[3]
q4 = diamonds_questions[4]

answer2, history2 = agent.solve(q2, verbose=True)
```

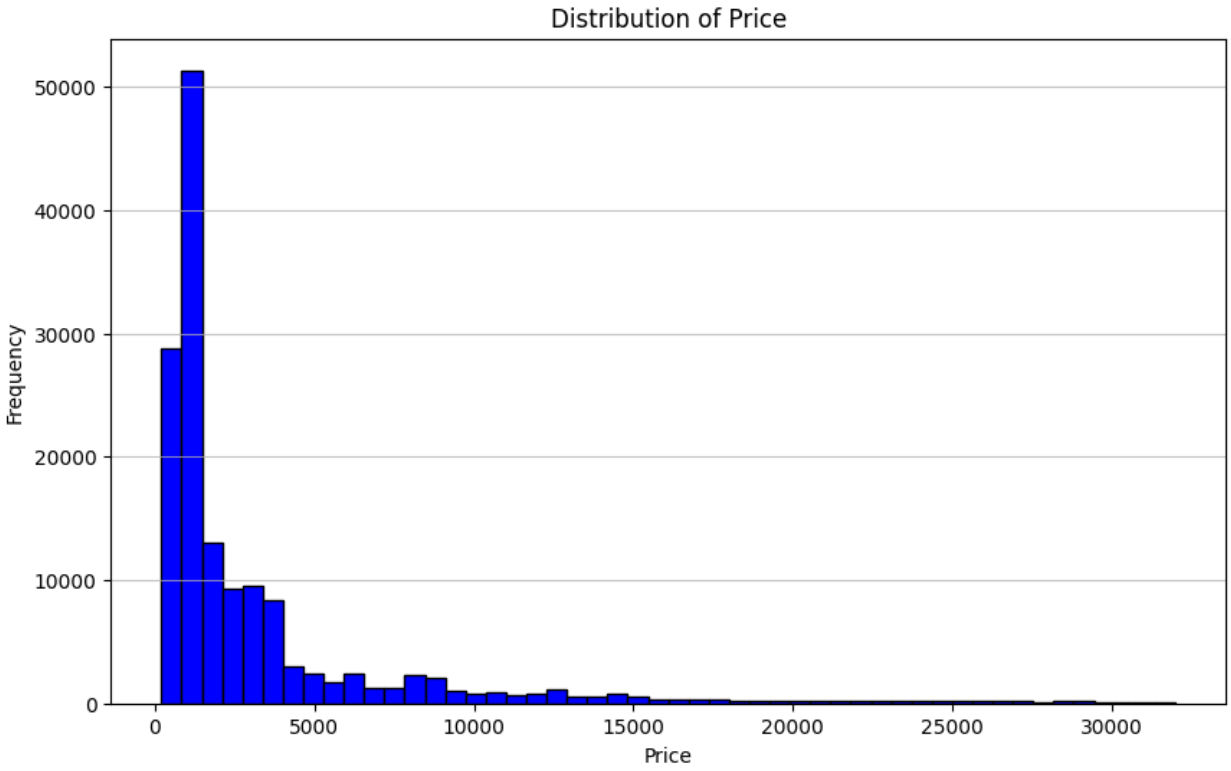
```
print("\nFINAL ANSWER Q2:")
print(answer2)

answer3, history3 = agent.solve(q3, verbose=True)
print("\nFINAL ANSWER Q3:")
print(answer3)

answer4, history4 = agent.solve(q4, verbose=True)
print("\nFINAL ANSWER Q4:")
print(answer4)
```

```
Train an Ordinary Least Squares (OLS) linear regression model to
predict 'price' using all other columns as features. Report the
average RMSE from 10-fold cross-validation on the validation set.
============================================================
PLANNER OUTPUT:
thought="I need to load the dataset, identify the features (all
columns except 'price'), and prepare the target variable 'price'.
Then, I will perform 10-fold cross-validation using KFold to compute
the RMSE." is_done=False response="Load the dataset from
'diamonds_selected.csv' and separate the features (all columns except
'price') and the target variable 'price'."
============================================================
CODER OUTPUT:
code="import pandas as pd\n\ndf =
pd.read_csv('diamonds_selected.csv')\nX = df.drop('price', axis=1)\ny
= df['price']\n\nprint(X.shape)\nprint(y.shape)"
============================================================
EXECUTOR OUTPUT:
(149871, 6)
(149871,)
============================================================
OBSERVER OUTPUT:
extracted_values={'dataset_shape': ' (149871, 6) ',
'target_variable_count': ' (149871,)'} warnings=[] error_category=None
next_step_hint="Explore the distribution of the target variable
'price' to understand its range and potential outliers."
============================================================
PLANNER OUTPUT:
thought="The next step is to load the dataset and examine the
distribution of the target variable 'price' to understand its range
and potential outliers, which will help in assessing the suitability
of the OLS model and the data for regression." is_done=False
response="Load the diamonds_selected.csv dataset and plot the
distribution of the 'price' column to examine its range and potential
outliers."
============================================================
CODER OUTPUT:
code='import pandas as pd\nimport matplotlib.pyplot as plt\n\n# Load
the diamonds_selected.csv dataset\nfile_path
```

```
= \'diamonds_selected.csv\'\ndf = pd.read_csv(file_path)\n\n# Plot the
distribution of the \'price\' column\nplt.figure(figsize=(10, 6))\
nplt.hist(df[\'price\'], bins=50, color=\'blue\',
edgecolor=\'black\')\nplt.title(\'Distribution of Price\')\
nplt.xlabel(\'Price\')\nplt.ylabel(\'Frequency\')\
nplt.grid(axis=\'y\', alpha=0.75)\n\n# Print summary statistics for
price\nprint(f"Mean price: {df[\'price\'].mean():.2f}")\
nprint(f"Median price: {df[\'price\'].median():.2f}")\
nprint(f"Standard deviation: {df[\'price\'].std():.2f}")\nprint(f"Min
price: {df[\'price\'].min()}")\nprint(f"Max price:
{df[\'price\'].max()}")\n\n# Show the plot\nplt.show()'
============================================================
```




```
EXECUTOR OUTPUT:
Mean price: 3303.92
Median price: 1302.00
Standard deviation: 4709.39
Min price: 200
Max price: 31996
============================================================
OBSERVER OUTPUT:
extracted_values={'mean_price': 3303.92, 'median_price': 1302.0,
'standard_deviation': 4709.39, 'min_price': 200, 'max_price': 31996}
warnings=[] error_category=None next_step_hint='Examine the
distribution plot to identify potential outliers, especially at the
```

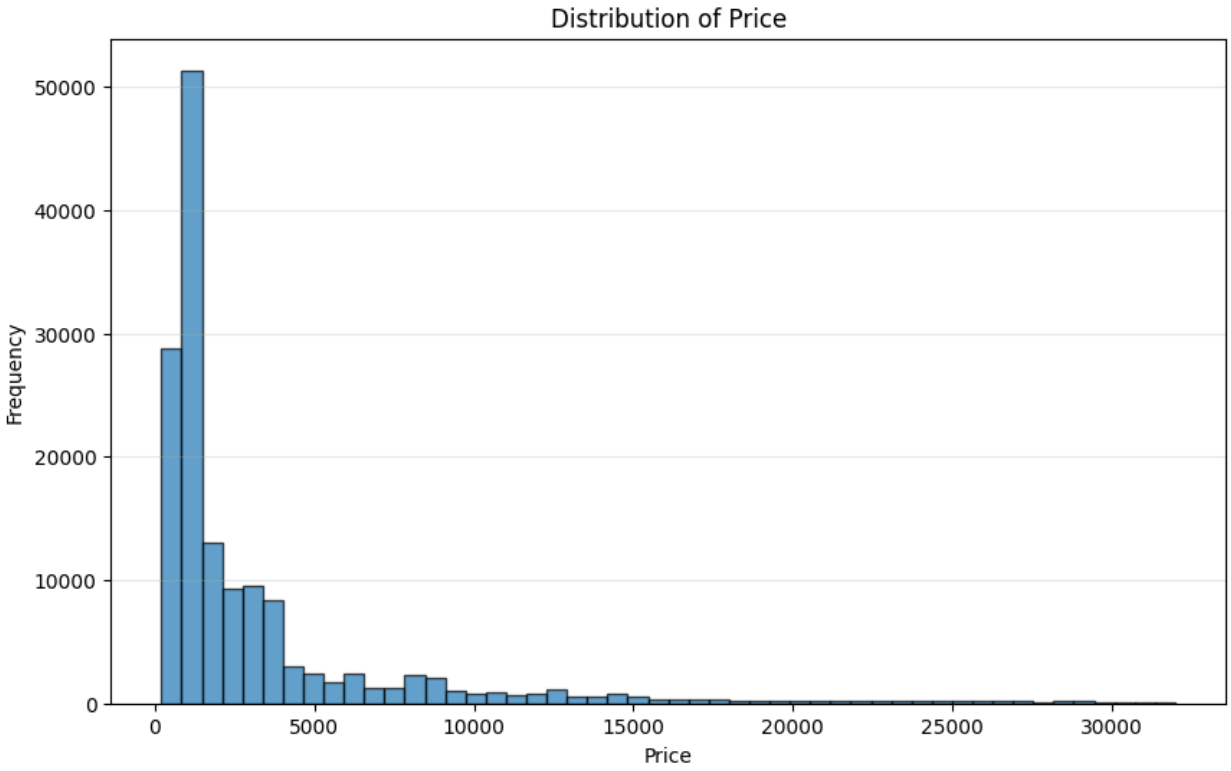
```
higher end of the price range.'
============================================================

FINAL ANSWER Q2:
Couldn't solve in 2 steps.
Train a Lasso regression model to predict 'price' using all other
columns as features. Use LassoCV to find the best regularization
parameter alpha via 10-fold cross-validation. Report the best alpha
and the average validation RMSE.
============================================================
PLANNER OUTPUT:
thought="The next step is to load the dataset and prepare the features
and target variable, separating all columns except 'price' as features
and 'price' as the target. Then, use LassoCV to find the optimal alpha
with 10-fold cross-validation." is_done=False response="Load the
dataset from 'diamonds_selected.csv' and separate the features (all
columns except 'price') and the target variable ('price')."
============================================================
CODER OUTPUT:
code="import pandas as pd\n\ndf =
pd.read_csv('diamonds_selected.csv')\nX = df.drop('price', axis=1)\ny
= df['price']\n\nprint(X.shape)\nprint(y.shape)"
============================================================
EXECUTOR OUTPUT:
(149871, 6)
(149871,)
============================================================
OBSERVER OUTPUT:
extracted_values={'dataset_shape': ' (149871, 6) ',
'target_variable_shape': ' (149871,) '} warnings=[]
error_category=None next_step_hint="Explore the distribution of the
target variable 'price' to understand its range and potential
outliers."
============================================================
PLANNER OUTPUT:
thought="The next step is to load the dataset and prepare the features
and target variable for modeling. Since the dataset has 6 columns and
the target is 'price', we need to select all columns except 'price' as
features." is_done=False response="Load the diamonds_selected.csv
dataset and separate the features (all columns except 'price') and the
target variable 'price'."
============================================================
CODER OUTPUT:
code="import pandas as pd\n\ndf =
pd.read_csv('diamonds_selected.csv')\n\nX = df.drop('price', axis=1)\
ny = df['price']\n\nprint(X.shape)\nprint(y.shape)"
============================================================
EXECUTOR OUTPUT:
(149871, 6)
```

```
(149871,)
============================================================
OBSERVER OUTPUT:
extracted_values={'dataset_shape': '149871, 6',
'target_variable_count': '149871'} warnings=[] error_category=None
next_step_hint="Proceed to explore the distribution of the target
variable 'price' or examine feature statistics."
============================================================

FINAL ANSWER Q3:
Couldn't solve in 2 steps.
Train a Ridge regression model to predict 'price' using all other
columns as features. Use RidgeCV to find the best regularization
parameter alpha via 10-fold cross-validation. Report the best alpha
and the average validation RMSE.
============================================================
PLANNER OUTPUT:
thought="I need to load the dataset and prepare the features and
target variable, excluding 'price' from features." is_done=False
response="Load the dataset from 'diamonds_selected.csv' and separate
the features (all columns except 'price') and the target variable
('price')."
============================================================
CODER OUTPUT:
code="import pandas as pd\n\ndf =
pd.read_csv('diamonds_selected.csv')\nX = df.drop('price', axis=1)\ny
= df['price']\nprint(X.shape)\nprint(y.shape)"
============================================================
EXECUTOR OUTPUT:
(149871, 6)
(149871,)
============================================================
OBSERVER OUTPUT:
extracted_values={'dataset_shape': ' (149871, 6) ',
'target_variable_shape': ' (149871, )'} warnings=[]
error_category=None next_step_hint="Explore the distribution of the
target variable 'price' to understand its range and potential
outliers."
============================================================
PLANNER OUTPUT:
thought="The next step is to load the dataset and examine the
distribution of the target variable 'price' to understand its range
and potential outliers, which can help in assessing the suitability of
Ridge regression and the expected scale of predictions." is_done=False
response="Load the diamonds_selected.csv dataset and plot the
distribution of the 'price' column to inspect its range and potential
outliers."
============================================================
CODER OUTPUT:
```

```
code='import pandas as pd\nimport matplotlib.pyplot as plt\n\n# Load
the diamonds_selected.csv dataset\nfile_path
= \'diamonds_selected.csv\'\ndf = pd.read_csv(file_path)\n\n# Plot the
distribution of the \'price\' column\nplt.figure(figsize=(10, 6))\
nplt.hist(df[\'price\'], bins=50, edgecolor=\'black\', alpha=0.7)\
nplt.title(\'Distribution of Price\')\nplt.xlabel(\'Price\')\
nplt.ylabel(\'Frequency\')\nplt.grid(axis=\'y\', alpha=0.3)\n\n# Print
summary statistics for price\nprint("Price range:",
df[\'price\'].min(), "to", df[\'price\'].max())\nprint("Mean price:",
df[\'price\'].mean())\nprint("Standard deviation of price:",
df[\'price\'].std())\n\n# Show the plot\nplt.show()'
============================================================
```




```
EXECUTOR OUTPUT:
Price range: 200 to 31996
Mean price: 3303.9154873190946
Standard deviation of price: 4709.385031997922
============================================================
OBSERVER OUTPUT:
extracted_values={'price_range': '200 to 31996', 'mean_price':
3303.9154873190946, 'std_dev_price': 4709.385031997922} warnings=[]
error_category=None next_step_hint='Consider identifying outliers in
the price distribution using statistical methods or visualizations
like box plots.'
============================================================
```

```
FINAL ANSWER Q4:
Couldn't solve in 2 steps.
```

```
def print_trace(history, title):
    print(f"\n===== {title} =====")
    for i, step in enumerate(history, 1):
        instruction, code, raw_output, observation = step
        print(f"\nSTEP {i}")
        print("Instruction:")
        print(instruction)
        print("\nCode:")
        print(code)
        print("\nRaw Output:")
        print(raw_output)
        print("\nObservation:")
        print(observation)
        print("-" * 80)

print_trace(history2, "Q2 TRACE")
print_trace(history3, "Q3 TRACE")
print_trace(history4, "Q4 TRACE")
```

```
===== Q2 TRACE =====

STEP 1
Instruction:
Load the dataset from 'diamonds_selected.csv' and separate the
features (all columns except 'price') and the target variable 'price'.

Code:
import pandas as pd

df = pd.read_csv('diamonds_selected.csv')
X = df.drop('price', axis=1)
y = df['price']

print(X.shape)
print(y.shape)

Raw Output:
(149871, 6)
(149871,)

Observation:
extracted_values={'dataset_shape': ' (149871, 6) ',
'target_variable_count': ' (149871,)'} warnings=[] error_category=None
next_step_hint="Explore the distribution of the target variable
'price' to understand its range and potential outliers."
--------------------------------------------------------------------------------
```

```
----------

STEP 2
Instruction:
Load the diamonds_selected.csv dataset and plot the distribution of
the 'price' column to examine its range and potential outliers.

Code:
import pandas as pd
import matplotlib.pyplot as plt

# Load the diamonds_selected.csv dataset
file_path = 'diamonds_selected.csv'
df = pd.read_csv(file_path)

# Plot the distribution of the 'price' column
plt.figure(figsize=(10, 6))
plt.hist(df['price'], bins=50, color='blue', edgecolor='black')
plt.title('Distribution of Price')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.grid(axis='y', alpha=0.75)

# Print summary statistics for price
print(f"Mean price: {df['price'].mean():.2f}")
print(f"Median price: {df['price'].median():.2f}")
print(f"Standard deviation: {df['price'].std():.2f}")
print(f"Min price: {df['price'].min()}")
print(f"Max price: {df['price'].max()}")

# Show the plot
plt.show()

Raw Output:
Mean price: 3303.92
Median price: 1302.00
Standard deviation: 4709.39
Min price: 200
Max price: 31996

Observation:
extracted_values={'mean_price': 3303.92, 'median_price': 1302.0,
'standard_deviation': 4709.39, 'min_price': 200, 'max_price': 31996}
warnings=[] error_category=None next_step_hint='Examine the
distribution plot to identify potential outliers, especially at the
higher end of the price range.'
----------------------------------------------------------------------
----------

STEP 3
```

```
Instruction:
Load the dataset from 'diamonds_selected.csv' and separate the
features (all columns except 'price') and the target variable
('price').

Code:
import pandas as pd

df = pd.read_csv('diamonds_selected.csv')
X = df.drop('price', axis=1)
y = df['price']

print(X.shape)
print(y.shape)

Raw Output:
(149871, 6)
(149871,)

Observation:
extracted_values={'dataset_shape': ' (149871, 6) ',
'target_variable_shape': ' (149871,) '} warnings=[]
error_category=None next_step_hint="Explore the distribution of the
target variable 'price' to understand its range and potential
outliers."
----------------------------------------------------------------------
----------

STEP 4
Instruction:
Load the diamonds_selected.csv dataset and separate the features (all
columns except 'price') and the target variable 'price'.

Code:
import pandas as pd

df = pd.read_csv('diamonds_selected.csv')

X = df.drop('price', axis=1)
y = df['price']

print(X.shape)
print(y.shape)

Raw Output:
(149871, 6)
(149871,)

Observation:
extracted_values={'dataset_shape': '149871, 6',
```

```
'target_variable_count': '149871'} warnings=[] error_category=None
next_step_hint="Proceed to explore the distribution of the target
variable 'price' or examine feature statistics."
--------------------------------------------------------------------
----------

STEP 5
Instruction:
Load the dataset from 'diamonds_selected.csv' and separate the
features (all columns except 'price') and the target variable
('price').

Code:
import pandas as pd

df = pd.read_csv('diamonds_selected.csv')
X = df.drop('price', axis=1)
y = df['price']
print(X.shape)
print(y.shape)

Raw Output:
(149871, 6)
(149871,)

Observation:
extracted_values={'dataset_shape': ' (149871, 6) ',
'target_variable_shape': ' (149871, )'} warnings=[]
error_category=None next_step_hint="Explore the distribution of the
target variable 'price' to understand its range and potential
outliers."
--------------------------------------------------------------------
----------

STEP 6
Instruction:
Load the diamonds_selected.csv dataset and plot the distribution of
the 'price' column to inspect its range and potential outliers.

Code:
import pandas as pd
import matplotlib.pyplot as plt

# Load the diamonds_selected.csv dataset
file_path = 'diamonds_selected.csv'
df = pd.read_csv(file_path)

# Plot the distribution of the 'price' column
plt.figure(figsize=(10, 6))
plt.hist(df['price'], bins=50, edgecolor='black', alpha=0.7)
```

```
plt.title('Distribution of Price')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.grid(axis='y', alpha=0.3)

# Print summary statistics for price
print("Price range:", df['price'].min(), "to", df['price'].max())
print("Mean price:", df['price'].mean())
print("Standard deviation of price:", df['price'].std())

# Show the plot
plt.show()

Raw Output:
Price range: 200 to 31996
Mean price: 3303.9154873190946
Standard deviation of price: 4709.385031997922

Observation:
extracted_values={'price_range': '200 to 31996', 'mean_price':
3303.9154873190946, 'std_dev_price': 4709.385031997922} warnings=[]
error_category=None next_step_hint='Consider identifying outliers in
the price distribution using statistical methods or visualizations
like box plots.'
----------------------------------------------------------------------
----------

===== Q3 TRACE =====

STEP 1
Instruction:
Load the dataset from 'diamonds_selected.csv' and separate the
features (all columns except 'price') and the target variable 'price'.

Code:
import pandas as pd

df = pd.read_csv('diamonds_selected.csv')
X = df.drop('price', axis=1)
y = df['price']

print(X.shape)
print(y.shape)

Raw Output:
(149871, 6)
(149871,)

Observation:
extracted_values={'dataset_shape': ' (149871, 6) ',
```

```
'target_variable_count': ' (149871,)'} warnings=[] error_category=None
next_step_hint="Explore the distribution of the target variable
'price' to understand its range and potential outliers."
----------------------------------------------------------------------
----------

STEP 2
Instruction:
Load the diamonds_selected.csv dataset and plot the distribution of
the 'price' column to examine its range and potential outliers.

Code:
import pandas as pd
import matplotlib.pyplot as plt

# Load the diamonds_selected.csv dataset
file_path = 'diamonds_selected.csv'
df = pd.read_csv(file_path)

# Plot the distribution of the 'price' column
plt.figure(figsize=(10, 6))
plt.hist(df['price'], bins=50, color='blue', edgecolor='black')
plt.title('Distribution of Price')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.grid(axis='y', alpha=0.75)

# Print summary statistics for price
print(f"Mean price: {df['price'].mean():.2f}")
print(f"Median price: {df['price'].median():.2f}")
print(f"Standard deviation: {df['price'].std():.2f}")
print(f"Min price: {df['price'].min()}")
print(f"Max price: {df['price'].max()}")

# Show the plot
plt.show()

Raw Output:
Mean price: 3303.92
Median price: 1302.00
Standard deviation: 4709.39
Min price: 200
Max price: 31996

Observation:
extracted_values={'mean_price': 3303.92, 'median_price': 1302.0,
'standard_deviation': 4709.39, 'min_price': 200, 'max_price': 31996}
warnings=[] error_category=None next_step_hint='Examine the
distribution plot to identify potential outliers, especially at the
higher end of the price range.'
```

```
--------------------------------------------------------------------------
----------

STEP 3
Instruction:
Load the dataset from 'diamonds_selected.csv' and separate the
features (all columns except 'price') and the target variable
('price').

Code:
import pandas as pd

df = pd.read_csv('diamonds_selected.csv')
X = df.drop('price', axis=1)
y = df['price']

print(X.shape)
print(y.shape)

Raw Output:
(149871, 6)
(149871,)

Observation:
extracted_values={'dataset_shape': ' (149871, 6) ',
'target_variable_shape': ' (149871,) '} warnings=[]
error_category=None next_step_hint="Explore the distribution of the
target variable 'price' to understand its range and potential
outliers."
--------------------------------------------------------------------------
----------

STEP 4
Instruction:
Load the diamonds_selected.csv dataset and separate the features (all
columns except 'price') and the target variable 'price'.

Code:
import pandas as pd

df = pd.read_csv('diamonds_selected.csv')

X = df.drop('price', axis=1)
y = df['price']

print(X.shape)
print(y.shape)

Raw Output:
(149871, 6)
```

```
(149871,)

Observation:
extracted_values={'dataset_shape': '149871, 6',
'target_variable_count': '149871'} warnings=[] error_category=None
next_step_hint="Proceed to explore the distribution of the target
variable 'price' or examine feature statistics."
----------------------------------------------------------------------
----------

STEP 5
Instruction:
Load the dataset from 'diamonds_selected.csv' and separate the
features (all columns except 'price') and the target variable
('price').

Code:
import pandas as pd

df = pd.read_csv('diamonds_selected.csv')
X = df.drop('price', axis=1)
y = df['price']
print(X.shape)
print(y.shape)

Raw Output:
(149871, 6)
(149871,)

Observation:
extracted_values={'dataset_shape': ' (149871, 6) ',
'target_variable_shape': ' (149871, )'} warnings=[]
error_category=None next_step_hint="Explore the distribution of the
target variable 'price' to understand its range and potential
outliers."
----------------------------------------------------------------------
----------

STEP 6
Instruction:
Load the diamonds_selected.csv dataset and plot the distribution of
the 'price' column to inspect its range and potential outliers.

Code:
import pandas as pd
import matplotlib.pyplot as plt

# Load the diamonds_selected.csv dataset
file_path = 'diamonds_selected.csv'
df = pd.read_csv(file_path)
```

```
# Plot the distribution of the 'price' column
plt.figure(figsize=(10, 6))
plt.hist(df['price'], bins=50, edgecolor='black', alpha=0.7)
plt.title('Distribution of Price')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.grid(axis='y', alpha=0.3)

# Print summary statistics for price
print("Price range:", df['price'].min(), "to", df['price'].max())
print("Mean price:", df['price'].mean())
print("Standard deviation of price:", df['price'].std())

# Show the plot
plt.show()

Raw Output:
Price range: 200 to 31996
Mean price: 3303.9154873190946
Standard deviation of price: 4709.385031997922

Observation:
extracted_values={'price_range': '200 to 31996', 'mean_price':
3303.9154873190946, 'std_dev_price': 4709.385031997922} warnings=[]
error_category=None next_step_hint='Consider identifying outliers in
the price distribution using statistical methods or visualizations
like box plots.'
----------------------------------------------------------------------
----------

===== Q4 TRACE =====

STEP 1
Instruction:
Load the dataset from 'diamonds_selected.csv' and separate the
features (all columns except 'price') and the target variable 'price'.

Code:
import pandas as pd

df = pd.read_csv('diamonds_selected.csv')
X = df.drop('price', axis=1)
y = df['price']

print(X.shape)
print(y.shape)

Raw Output:
(149871, 6)
```

```
(149871,)

Observation:
extracted_values={'dataset_shape': ' (149871, 6) ',
'target_variable_count': ' (149871,)'} warnings=[] error_category=None
next_step_hint="Explore the distribution of the target variable
'price' to understand its range and potential outliers."
--------------------------------------------------------------------
----------

STEP 2
Instruction:
Load the diamonds_selected.csv dataset and plot the distribution of
the 'price' column to examine its range and potential outliers.

Code:
import pandas as pd
import matplotlib.pyplot as plt

# Load the diamonds_selected.csv dataset
file_path = 'diamonds_selected.csv'
df = pd.read_csv(file_path)

# Plot the distribution of the 'price' column
plt.figure(figsize=(10, 6))
plt.hist(df['price'], bins=50, color='blue', edgecolor='black')
plt.title('Distribution of Price')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.grid(axis='y', alpha=0.75)

# Print summary statistics for price
print(f"Mean price: {df['price'].mean():.2f}")
print(f"Median price: {df['price'].median():.2f}")
print(f"Standard deviation: {df['price'].std():.2f}")
print(f"Min price: {df['price'].min()}")
print(f"Max price: {df['price'].max()}")

# Show the plot
plt.show()

Raw Output:
Mean price: 3303.92
Median price: 1302.00
Standard deviation: 4709.39
Min price: 200
Max price: 31996

Observation:
extracted_values={'mean_price': 3303.92, 'median_price': 1302.0,
```

```
'standard_deviation': 4709.39, 'min_price': 200, 'max_price': 31996}
warnings=[] error_category=None next_step_hint='Examine the
distribution plot to identify potential outliers, especially at the
higher end of the price range.'
-------------------------------------------------------------------
----------

STEP 3
Instruction:
Load the dataset from 'diamonds_selected.csv' and separate the
features (all columns except 'price') and the target variable
('price').

Code:
import pandas as pd

df = pd.read_csv('diamonds_selected.csv')
X = df.drop('price', axis=1)
y = df['price']

print(X.shape)
print(y.shape)

Raw Output:
(149871, 6)
(149871,)

Observation:
extracted_values={'dataset_shape': ' (149871, 6) ',
'target_variable_shape': ' (149871,) '} warnings=[]
error_category=None next_step_hint="Explore the distribution of the
target variable 'price' to understand its range and potential
outliers."
-------------------------------------------------------------------
----------

STEP 4
Instruction:
Load the diamonds_selected.csv dataset and separate the features (all
columns except 'price') and the target variable 'price'.

Code:
import pandas as pd

df = pd.read_csv('diamonds_selected.csv')

X = df.drop('price', axis=1)
y = df['price']

print(X.shape)
print(y.shape)
```

```
Raw Output:
(149871, 6)
(149871,)

Observation:
extracted_values={'dataset_shape': '149871, 6',
'target_variable_count': '149871'} warnings=[] error_category=None
next_step_hint="Proceed to explore the distribution of the target
variable 'price' or examine feature statistics."
----------------------------------------------------------------------
----------

STEP 5
Instruction:
Load the dataset from 'diamonds_selected.csv' and separate the
features (all columns except 'price') and the target variable
('price').

Code:
import pandas as pd

df = pd.read_csv('diamonds_selected.csv')
X = df.drop('price', axis=1)
y = df['price']
print(X.shape)
print(y.shape)

Raw Output:
(149871, 6)
(149871,)

Observation:
extracted_values={'dataset_shape': ' (149871, 6) ',
'target_variable_shape': ' (149871, )'} warnings=[]
error_category=None next_step_hint="Explore the distribution of the
target variable 'price' to understand its range and potential
outliers."
----------------------------------------------------------------------
----------

STEP 6
Instruction:
Load the diamonds_selected.csv dataset and plot the distribution of
the 'price' column to inspect its range and potential outliers.

Code:
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Load the diamonds_selected.csv dataset
file_path = 'diamonds_selected.csv'
df = pd.read_csv(file_path)

# Plot the distribution of the 'price' column
plt.figure(figsize=(10, 6))
plt.hist(df['price'], bins=50, edgecolor='black', alpha=0.7)
plt.title('Distribution of Price')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.grid(axis='y', alpha=0.3)

# Print summary statistics for price
print("Price range:", df['price'].min(), "to", df['price'].max())
print("Mean price:", df['price'].mean())
print("Standard deviation of price:", df['price'].std())

# Show the plot
plt.show()

Raw Output:
Price range: 200 to 31996
Mean price: 3303.9154873190946
Standard deviation of price: 4709.385031997922

Observation:
extracted_values={'price_range': '200 to 31996', 'mean_price':
3303.9154873190946, 'std_dev_price': 4709.385031997922} warnings=[]
error_category=None next_step_hint='Consider identifying outliers in
the price distribution using statistical methods or visualizations
like box plots.'
--------------------------------------------------------------------
----------
```